MCP-Enriched Faculty Assistant for Timetable and Room Queries: Review, Approaches and Open Research Problems

Abstract

Faculty members and students routinely need quick answers to questions such as "Where is my 10 am lecture on Thursday?", "Which rooms are free tomorrow afternoon?" or "Has my class been moved?". In most institutions these answers are scattered across timetable spreadsheets, learning-management systems, room-booking portals and notice boards, and are accessed through rigid search forms. Large language model (LLM) assistants can answer such questions in natural language, but on their own they hallucinate, cannot see live institutional data, and are hard to integrate with each backend system. The Model Context Protocol (MCP), an open standard for connecting LLM applications to external tools and data, offers a uniform way to expose timetable, room and booking services to an assistant. This paper presents a systematic review of the concepts and approaches relevant to building an MCP-enriched faculty assistant: classical timetabling systems, conversational interfaces for academic information, tool-augmented LLMs, retrieval-augmented generation, and MCP itself. We describe a reference architecture, compare design alternatives, define evaluation measures suited to timetable and room queries, and identify open research problems, including data freshness, ambiguity resolution, access control, prompt-injection security and evaluation benchmarks. We recommend a tool-grounded, read-mostly assistant with strict permission boundaries as the practical starting point.

Keywords: Model Context Protocol; large language models; timetable; room allocation; conversational assistant; tool use; retrieval-augmented generation; academic information systems

1. Introduction

University timetables are among the most frequently consulted and most frequently changing pieces of institutional information. A single department may run hundreds of sessions per week across shared lecture halls, laboratories and seminar rooms. Rooms are swapped, sessions are rescheduled, staff are on leave, and examinations temporarily displace regular teaching. Faculty members who need to answer a simple question often have to open a spreadsheet, a PDF export, or a scheduling portal whose interface was designed for administrators rather than for quick lookups.

Conversational assistants promise to remove this friction. A faculty member should be able to type or say "Is Lab 3 free on Friday after 2?" and receive a correct, verifiable answer. Early chatbots for academic information relied on hand-written intents and templates, which are brittle and costly to maintain. LLMs handle free-form language far better, but they introduce two new problems. First, an LLM has no knowledge of this week's timetable, so an unaided model will either refuse or fabricate. Second, connecting a model to each data source has traditionally required custom, one-off integrations.

The Model Context Protocol (MCP) addresses the second problem. It defines a standard client-server interface through which an LLM application can discover and call *tools*, read *resources* and use *prompt templates* exposed by independent servers [1,2]. A timetable database, a room-booking system and a staff directory can each be wrapped as an MCP server once and then used by any MCP-compatible assistant.

The contributions of this work are as follows:

a. A survey of the main approaches to timetable and room information access, from classical timetabling systems to LLM agents.

b. A reference architecture for an MCP-enriched faculty assistant, including the server-side tools needed for timetable and room queries.

c. A comparison of design alternatives (rule-based, retrieval-only, fine-tuned and tool-grounded) with their strengths and drawbacks.

d. Evaluation measures appropriate to this domain, with formulae.

e. A list of open research problems and recommended next steps.

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 gives background on the problem and on MCP and presents the architecture. Section 4 defines performance measures. Section 5 describes the candidate techniques. Section 6 compares them. Section 7 discusses open problems, Section 8 covers AI platform engineering considerations and Section 9 concludes.

2. Related Work

The literature relevant to this project falls into four strands. Table 1 summarises how this review differs from earlier surveys.

Automated timetabling. Schaerf [3] surveyed automated timetabling, distinguishing school, course and examination timetabling and the main solution families. Lewis [4] reviewed metaheuristic techniques for university timetabling, including simulated annealing, tabu search and genetic algorithms. These works focus on *constructing* a feasible timetable under hard and soft constraints. They do not address how people later *query* the result, which is the concern here.

Task-oriented dialogue. Conversational systems for booking and information lookup have long been studied, with datasets such as MultiWOZ [5] driving work on slot filling and state tracking. Such pipelines need intent and slot annotations and do not generalise easily to new institutions or phrasing.

Tool-augmented language models. ReAct [6] interleaves reasoning steps with actions so a model can consult external sources. Toolformer [7] showed models can learn when to call tools, Gorilla [8] focused on accurate API calls, and ToolLLM [9] scaled tool use to thousands of real APIs. These works show that an LLM can answer from live data by calling functions, which is exactly what timetable queries require.

Retrieval-augmented generation (RAG). RAG [10] grounds generation in retrieved documents. It suits unstructured sources such as handbooks, regulations and notices, but is less natural for highly structured, rapidly changing data such as a week-by-week room schedule, where an exact query is better than a similarity search.

MCP and its security. MCP was introduced in late 2024 [1] and is specified publicly [2]. An early landscape study catalogued its architecture, ecosystem and security threats [11], and the broader literature on indirect prompt injection [12] explains why tool outputs must be treated as untrusted.

Table 1. Summary of previous work and the position of this review.

Work	Timetable construction	Dialogue / NL access	Tool use / LLM agents	Standard protocol (MCP)	Evaluation measures	Architecture
Schaerf [3]	✓					
Lewis [4]	✓				✓	
MultiWOZ-style dialogue [5]		✓			✓	✓
ReAct, Toolformer, Gorilla, ToolLLM [6–9]			✓		✓	
MCP landscape study [11]			✓	✓		✓
This review	✓	✓	✓	✓	✓	✓

This review differs by treating the *query and access layer* on top of an existing timetable, and by focusing on MCP as the integration mechanism.

3. Background

3.1 The problem: timetable and room queries

A timetable is a set of *sessions*. Each session links a course, one or more lecturers, a student group, a room, a day and a time slot, and possibly a week pattern (weekly, odd weeks, one-off). Room data adds capacity, building, equipment (projector, lab benches, accessibility features) and booking status. Typical faculty queries fall into the following classes:

- **Lookup:** "When and where is CSC 301 this week?"
- **Availability:** "Which rooms with at least 60 seats are free on Wednesday 2–4 pm?"
- **Conflict check:** "Do I have anything clashing with a meeting at 11 on Monday?"
- **Change notification:** "Has any of my sessions moved since yesterday?"
- **Comparative or compound:** "Find the nearest free room to Block B for my 90-minute tutorial."
- **Action (optional):** "Request Room 204 for a make-up class on Friday."

These queries are short, but they depend on exact, current data. A wrong room number is worse than no answer, which makes this a precision-critical domain.

3.2 The Model Context Protocol

MCP follows a client-server design. A *host* application (the assistant) contains an MCP *client* that connects to one or more MCP *servers*. Each server exposes three kinds of capability [2]:

- **Tools:** callable functions with typed input schemas (for example `find_free_rooms`).
- **Resources:** readable data identified by URIs (for example a room's equipment list).
- **Prompts:** reusable templates that guide common interactions.

The client discovers capabilities at connection time, so adding a new data source does not require changing the assistant itself. This reduces the integration burden from one custom connector per model and system pair to one MCP server per system.

3.3 Reference architecture

Figure 1 (described in text) shows the proposed layers.

1. **Interface layer:** web chat, mobile app or messaging channel used by the faculty member.
2. **Assistant (host) layer:** an LLM with a system prompt defining scope, tone and refusal rules, plus the MCP client and a conversation memory.

3. **MCP server layer:** independent servers, each wrapping one backend.
    - *Timetable server:* `get_sessions(lecturer, course, date_range)`, `get_changes(since)`.
    - *Room server:* `find_free_rooms(date, start, end, min_capacity, features)`, `get_room_details(room_id)`.
    - *Calendar server:* `get_personal_events(user, date_range)` for conflict checks.
    - *Directory server:* `resolve_person(name)`, `resolve_course(code_or_title)`.
    - *Booking server (optional, write access):* `request_room(...)`, gated by confirmation.
4. **Data layer:** the institutional timetable database or scheduling system, room inventory, and calendar provider.
5. **Governance layer:** authentication, role-based permissions, rate limiting and audit logging, applied at the MCP server boundary.

A typical query flow is: the user asks a question; the model decides which tools are needed; the client calls the servers; structured results return; the model composes an answer that cites the returned data (room, time, date) and states when the data was fetched.

3.4 Data preparation

Before tools can be reliable, the underlying data must be normalised. Important steps are:

- **Canonical identifiers** for courses, rooms, buildings and staff, with an alias table ("LT1", "Lecture Theatre 1", "Main Hall 1").
- **Time normalisation** to a single time zone and explicit handling of week patterns, semester boundaries and public holidays.
- **Change tracking** through versioned records or an event log so that "what changed" is answerable.
- **Schema validation** so that servers return typed, bounded results rather than raw database dumps.

4. Analysis: Performance Evaluation Measures

As with spam filtering, a single accuracy figure is not enough, because errors have unequal costs. Telling a lecturer that a free room is free when it is booked (a false positive for availability) is more damaging than failing to suggest a room that was actually free (a false negative). Suitable measures are given below.

Task success rate (TSR). The share of queries answered correctly end to end.

TSR = N_correct / N_total (1)

Answer precision and recall for set-valued queries (such as "list free rooms"). Let R be the set of rooms returned and G the ground-truth set.

Precision = |R ∩ G| / |R| (2)

Recall = |R ∩ G| / |G| (3)

F1 = 2 · Precision · Recall / (Precision + Recall) (4)

Cost-weighted error. Following the cost-sensitive idea used in spam evaluation, a false availability claim can be weighted λ times more than a missed room.

WErr = (λ · N_falseAvail + N_missed) / N_total (5)

Tool-call accuracy. Whether the model selected the right tool and supplied valid arguments.

ToolAcc = N_correctCalls / N_calls (6)

Entity resolution accuracy. The share of person, course and room mentions mapped to the correct canonical identifier.

Hallucination rate. The share of answers containing a room, time or name that does not appear in any tool result.

Freshness. The age of the data behind an answer, measured from last successful sync to answer time. A target (for example under five minutes) should be set.

Latency and cost. Median and 95th-percentile response time, number of tool calls per query, and tokens consumed.

Refusal quality. The proportion of out-of-scope or unauthorised requests that are correctly declined, together with the proportion of valid requests wrongly declined.

User-centred measures. Task completion time compared with the existing portal, plus a satisfaction scale such as SUS.

A publicly available benchmark specific to timetable queries does not yet exist to our knowledge. Table 2 therefore proposes the dataset types a project of this kind would need to build.

Table 2. Proposed evaluation datasets.

Dataset	Content	Purpose
Query set	Several hundred natural-language faculty questions with gold answers, covering all classes in Section 3.1	TSR, ToolAcc

Dataset	Content	Purpose
Alias set	Variant spellings and abbreviations of rooms, courses and staff	Entity resolution
Change log replay	A recorded semester of schedule edits	Freshness and change queries
Adversarial set	Prompt-injection strings placed in session titles, notes and room descriptions	Security
Out-of-scope set	Requests for other users' data or for unsupported actions	Refusal quality

5. Materials and Methods: Candidate Approaches

5.1 Rule-based and form-based search

The incumbent approach is a search form or a keyword matcher with hand-written patterns. It is predictable and fast, but each new phrasing needs a new rule, and compound questions are not supported. Like the knowledge-engineering filters described in the spam literature, the maintenance cost grows with every change.

5.2 Intent classification with slot filling

A classifier maps a query to an intent (lookup, availability, and so on) and extracts slots (course, date, time, capacity). This is the classical task-oriented dialogue pipeline [5]. It is accurate within its trained scope and cheap to run, but needs labelled data and handles unseen phrasing and multi-step questions poorly.

5.3 Retrieval-augmented generation over exported timetables

Timetable exports (PDF, spreadsheet, web pages) are chunked, embedded and retrieved to ground an LLM's answer [10]. This is quick to prototype and works on unstructured notices. Its weaknesses for this domain are staleness between re-indexing runs, poor handling of exact filters such as "capacity at least 60 and free 2–4 pm", and the risk that similarity search returns a neighbouring but wrong row.

5.4 Text-to-SQL or text-to-query

The model translates the question into a database query. It is expressive and exact when the SQL is correct. Risks include malformed or unsafe queries, dependence on schema knowledge and the need for read-only sandboxing. This can be offered as one MCP tool,

but exposing a general query tool widens the attack surface compared with narrow, purpose-built tools.

5.5 Tool-grounded LLM agent over MCP (proposed)

The assistant is given narrowly scoped MCP tools such as those in Section 3.3 and reasons over their results, using a ReAct-style loop [6]. Because each tool has a typed schema, the model's task is reduced to choosing tools and arguments, which tool-use research suggests is more reliable than free-form generation of answers [7,8,9]. The data stays in the source system and is read at question time, which addresses freshness. Permissions and logging are enforced in the server rather than in the prompt.

A simplified procedure is:

```
Input: user query q, user identity u
1: Resolve identity and role of u
2: Plan: identify entities (people, courses, rooms, dates) in q
3: for each ambiguous entity: call resolve_* tools; if still ambiguous, ask a
clarifying question
4: Call timetable / room / calendar tools with validated arguments
5: If a result is empty or an error occurs, retry with relaxed arguments or
report the limitation
6: Compose answer using only values present in tool results
7: Attach data timestamp and offer follow-ups
8: Log query, tool calls and outcome
```

5.6 Hybrid designs

A practical system combines approaches. Structured data (sessions, rooms, bookings) is served through MCP tools, while unstructured material (examination regulations, room-use policy, notices) is served through a retrieval resource. A lightweight intent router can send trivial lookups straight to a fixed tool path to reduce latency and cost.

5.7 Optimisation back-ends

If the assistant is later asked for suggestions, such as "propose a new slot for this class", it can call a constraint solver or metaheuristic scheduler [3,4] as another MCP tool. The LLM handles language and orchestration, while the solver guarantees constraint satisfaction. This separation avoids asking a language model to do combinatorial optimisation, which it does unreliably.

6. Comparative Study of Approaches

Because no timetable-query benchmark exists, Table 3 gives a qualitative comparison based on the properties of each approach as described in the literature, not on measured results from this project.

Table 3. Qualitative comparison of candidate approaches.

Approach	Strengths	Drawbacks	Freshness	Handles compound queries	Integration effort
Rule / form search	Predictable, fast	Brittle, high maintenance	Depends on source	Poor	Low to medium
Intent + slot filling	Accurate in scope, cheap	Needs labelled data, narrow	Good	Limited	Medium
RAG over exports	Fast to prototype, handles documents	Stale index, weak exact filtering	Poor to medium	Medium	Low
Text-to-SQL	Expressive, exact when correct	Unsafe or invalid queries, schema coupling	Good	Good	Medium
MCP tool-grounded agent	Live data, typed tools, reusable servers, auditable	Model errors in tool choice, security surface, per-call latency	Good	Good	Medium (one server per backend)
Hybrid (MCP + retrieval)	Covers structured and unstructured needs	More components to maintain	Good	Good	Medium to high

The main observations are:

- Approaches that read live data at question time are better suited to timetables than those relying on periodically rebuilt indexes.
- Narrow, well-described tools outperform broad ones in controllability and security, at the cost of more server development.
- Reported gaps in the tool-use literature, such as incorrect arguments and unnecessary calls [8,9], translate directly into the risk of wrong dates or wrong rooms and must be mitigated by argument validation and by checking results.
- The same limitation seen in the spam-filtering review applies here: studies that report a single metric on a private dataset are difficult to compare. Any project in this area should publish its query set and several measures.

7. Discussion and Open Research Problems

Security of the assistant. The spam-filtering review highlighted adversarial attacks on learners [Dada et al.]. The analogue here is prompt injection: text stored in a session title, a room note or an email could contain instructions aimed at the model [12]. MCP servers also introduce risks such as over-privileged tools, malicious or compromised servers and tool-description poisoning [11]. Mitigations include treating all tool output as data, least-privilege scopes, allow-listed servers, human confirmation for write actions and audit logs. How to evaluate these defences systematically remains open.

Concept drift in schedules. Timetables change every semester, and sometimes every day. A system that caches or learns from past schedules can become wrong without notice. Detecting stale data, propagating change events to the assistant and communicating uncertainty are unresolved design questions.

Ambiguity and entity resolution. "The big lab", "Dr. Musa" (several lecturers) and "Monday" (this or next) are routine. Choosing between asking for clarification and making a confident guess has a cost on both sides, and good policies are not well studied for this setting.

Access control and privacy. Faculty may see their own teaching load, while student-group data and other staff members' calendars may be restricted. Identity must flow from the user through the host and into each server, rather than being asserted by the model. Standardised patterns for per-user authorisation across MCP servers are still developing.

Reliability of tool use. Models can choose the wrong tool, omit a filter or misread a time zone. Techniques such as schema constraints, deterministic post-checks (for example verifying that a returned room is not double-booked) and answer-from-evidence prompting need empirical comparison on timetable data.

Benchmarks and evaluation. No shared benchmark covers academic scheduling queries, and the Section 4 measures lack reference results. Building and releasing such a benchmark, including adversarial and multilingual cases, would be a valuable contribution.

Multilingual and voice access. Institutions often serve users in several languages and many queries are made on mobile devices while walking between buildings. Robustness to code-switching, spoken input and accent variation is an open area.

Write actions. Allowing the assistant to request or change room bookings brings concurrency, authorisation and accountability issues. A conservative position is to start read-only and add writes behind explicit confirmation.

Cost and latency. Multi-step tool use can be slow and expensive for a simple lookup. Routing simple intents to deterministic paths and caching safe, short-lived results are promising but need evaluation.

Integration with solvers. Combining an LLM front end with a constraint-based scheduler for "suggest an alternative" queries is attractive, yet explaining solver output in natural language and handling infeasible requests remains underexplored.

In summary, the open research problems are:

- Effective defences against prompt injection and malicious MCP servers in an academic setting.
- Handling schedule drift and communicating data freshness.
- Reliable disambiguation of people, courses and rooms.
- Per-user authorisation across multiple MCP servers.
- A public benchmark and agreed metrics for timetable and room queries.
- Safe, auditable write operations.
- Multilingual and voice-based access.
- Cost-efficient routing between deterministic paths and LLM reasoning.
- LLM and constraint-solver integration for schedule suggestions.

8. AI Platform Engineering Considerations

An assistant that works in a demonstration is not yet a platform service. This section describes what is needed to run the MCP-enriched faculty assistant reliably, securely and affordably as a shared institutional service.

8.1 Deployment architecture

The system is split into independently deployable components, each in its own container:

1. **Assistant service (stateless):** hosts the LLM client, the MCP client and the system prompt. Conversation state is kept in an external store so that any replica can serve any request.
2. **Model gateway:** a single internal endpoint through which all LLM calls pass. It handles provider credentials, quotas, retries, logging and model selection.
3. **MCP servers:** one container per backend (timetable, rooms, calendar, directory, optional booking). Each is deployed, scaled and versioned on its own.
4. **Identity and policy layer:** validates the user's token, passes the user identity to each server and enforces role-based permissions.
5. **Data and cache layer:** the institutional database (read replica for queries), a short-lived cache and a session store.
6. **Observability stack:** collects traces, metrics and logs from every component.

The assistant should reach MCP servers only through this gateway and policy layer, so that new servers can be added or retired without changing the assistant code.

8.2 LLMOps and CI/CD

Prompts, tool descriptions and tool schemas change behaviour as much as code does, so they are treated as versioned artefacts.

- **Version control** for the system prompt, tool schemas and server code, with release tags.
- **Automated regression runs:** on every change, the query set from Table 2 is replayed against the assistant and compared with the previous release on task success rate, tool-call accuracy and hallucination rate.
- **Release gates:** a change is promoted only if the scores do not fall below agreed thresholds and the adversarial (prompt-injection) set passes.
- **Staged rollout:** a canary release to a small group of faculty, with rollback to the previous prompt and schema version when quality drops.
- **Contract tests** between the assistant and each MCP server, so a schema change in the timetable database is caught before it breaks tool calls.

8.3 Observability

Each user query receives a trace identifier that follows it through the model gateway and every tool call. The platform records:

- Latency per step (model, tool, database) and end to end.
- Tokens used and estimated cost per query.
- Tool-call success, error and timeout rates per server.
- The data timestamp used in each answer, to monitor freshness.
- User feedback (thumbs up or down) linked to the trace, so poor answers can be replayed and added to the test set.

Logs must avoid storing unnecessary personal data. Retention periods and access to traces should follow institutional policy.

8.4 Scalability and cost control

- **Caching:** results that change slowly (room details, course lists) are cached for a long time, and timetable results for a short, explicit time.
- **Routing:** simple, high-frequency queries ("my timetable today") go to a deterministic path that calls one tool without a model reasoning loop. Complex or ambiguous queries go to the full agent.

- **Model tiering:** a smaller, cheaper model handles routine lookups, and a stronger model is used when the query is compound or the first attempt fails validation.
- **Limits:** a cap on tool calls and tokens per query, and per-user rate limits, to prevent runaway loops and abuse.
- **Horizontal scaling:** the assistant and MCP servers scale on request volume, with peaks expected at the start of the semester and the day before examinations.

8.5 Reliability and fallback behaviour

The assistant depends on several systems, so failure handling must be designed rather than left to the model.

- **Timeouts and retries** with limits on each tool call.
- **Circuit breakers** that stop calling a failing server for a short period.
- **Stale-data policy:** if the latest sync is older than the freshness target, the answer states the data's age or declines to give an availability answer.
- **Graceful degradation:** if the room server is down, the assistant can still answer timetable lookups and says which part it cannot confirm.
- **Honest refusal:** when results are empty or contradictory, the assistant says it cannot confirm the answer and points to the official portal or the timetable office.

8.6 Model gateway and portability

Because tools are exposed through MCP and models are reached through a gateway, the LLM provider can be changed without rewriting the integrations. This allows the same query set to be run against several models for comparison, supports fallback to a second provider during an outage, and reduces vendor lock-in. The evaluation harness should therefore record the model name and version with every result.

8.7 Platform security and governance

- Secrets are held in a secrets manager, never in prompts or tool descriptions.
- Only an allow-list of reviewed MCP servers can be connected.
- Each server runs with the least privilege needed (read-only database account for query tools).
- Write actions require user confirmation and produce an audit record.
- Personal data minimisation, retention limits and the user's right to know that they are talking to an AI system are documented as policy.

Table 4. Platform concerns mapped to practices.

Concern	Practice	Measure
Quality drift after prompt or schema changes	Versioned artefacts, regression suite, release gates	Task success rate, tool-call accuracy per release
Slow or costly answers	Caching, routing, model tiering, call limits	p95 latency, tokens and cost per query
Backend outage	Timeouts, circuit breakers, graceful degradation	Availability, share of answers with fallback notice
Stale answers	Sync monitoring, stale-data policy	Data age at answer time
Injection and misuse	Allow-listed servers, least privilege, confirmations, audit logs	Adversarial set pass rate, blocked unauthorised calls
Provider lock-in or outage	Model gateway, portable tool layer	Time to switch model, cross-model score gap

9. Conclusion

This paper reviewed approaches relevant to an MCP-enriched faculty assistant for timetable and room queries. Classical timetabling research tells us how schedules are built, task-oriented dialogue and RAG show how users can query information in natural language, and tool-augmented LLMs show how a model can answer from live systems. MCP supplies the standard interface that lets these elements be assembled without bespoke connectors for each backend. We proposed a layered architecture with narrow, typed tools, evaluation measures that reflect the unequal cost of errors, and a qualitative comparison of alternatives. We recommend starting with a read-only, tool-grounded assistant with server-side access control, evaluating it against a purpose-built query set, and extending to bookings and schedule suggestions only once security and reliability have been demonstrated. We hope this paper serves as a starting point for empirical work on this practical and under-studied problem.

Suggested Next Steps for the Project

1. Inventory the data sources (timetable system, room inventory, calendars) and define the canonical identifiers.
2. Build two or three MCP servers (timetable, rooms, directory) with read-only tools.

3. Create the Table 2 query set and baseline against the current portal.
4. Run the assistant, measure the Section 4 metrics, and iterate on tool descriptions and validation.
5. Add the security tests before any write capability is considered.

References

Note: please verify bibliographic details (volume, pages, URLs) against the original sources before submission.

[1] Anthropic, "Introducing the Model Context Protocol," Anthropic announcement, Nov. 2024.

[2] Model Context Protocol specification, https://modelcontextprotocol.io.

[3] A. Schaerf, "A survey of automated timetabling," Artificial Intelligence Review, vol. 13, pp. 87–127, 1999.

[4] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," OR Spectrum, vol. 30, pp. 167–190, 2008.

[5] P. Budzianowski et al., "MultiWOZ: A large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling," EMNLP, 2018.

[6] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," ICLR, 2023.

[7] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," NeurIPS, 2023.

[8] S. G. Patil et al., "Gorilla: Large language model connected with massive APIs," arXiv:2305.15334, 2023.

[9] Y. Qin et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," arXiv:2307.16789, 2023.

[10] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," NeurIPS, 2020.

[11] X. Hou et al., "Model Context Protocol (MCP): Landscape, security threats, and future research directions," arXiv:2503.23278, 2025.

[12] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv:2302.12173, 2023.

[13] E. G. Dada et al., "Machine learning for email spam filtering: review, approaches and open research problems," Heliyon, vol. 5, e01802, 2019 (structural model for this paper).